

Computer Vision

Introduction to Images and Formats

Section 1: Theoretical Foundations

1.1 Introduction to Digital Image Representation

- **What is an Image?** An image is a numerical matrix or a grid of numbers that represents the intensity or color of each point (pixel) in the picture.
- **Pixels:** The smallest building block of a digital image. They act as a mosaic, containing intensity and color information to create a complete visual representation.
- **Why Images Matter:** In Computer Vision, images allow systems to detect objects, recognize faces, track movements, read text (OCR), and facilitate autonomous driving.
- **Coordinate Systems:** Digital images are indexed using (x, y) coordinates. In most frameworks, (0,0) is the top-left corner.
- **Resolution and Matrix Representation:** High-resolution images contain more pixels, allowing for finer detail but requiring higher computational resources.

1.2 Image Formats and Metadata

- **JPEG (Joint Photographic Experts Group):** Lossy compression, small file sizes, no transparency support. Ideal for web and massive datasets.
- **PNG (Portable Network Graphics):** Lossless compression, supports alpha channels (transparency). Standard for high-precision imaging.
- **TIFF (Tagged Image File Format):** Archival quality, flexible metadata, but very large file sizes. Used in professional photography.
- **BMP (Bitmap):** Simple, uncompressed format. Rarely used in production due to lack of optimization.
- **EXIF Metadata:** Exchangeable Image File Format. Stores camera settings, GPS, and time. Crucial for forensics and geographic mapping.

1.3 Color Models and Bit Depth

- **Grayscale:** Each pixel is represented by a single intensity value, typically from 0 (Black) to 255 (White).
- **RGB Model:** An additive model where Red, Green, and Blue channels are combined. Each channel usually has 8 bits.
- **Bit Depth:** The number of bits used to represent intensity. 8-bit = 256 levels; 24-bit = 16.7 million colors.
- **1-bit Depth:** Binary images (only 0 and 1) used for masks and OCR processing.
- **16/32-bit Depth:** Used in high-end sensors and HDR imaging for professional color grading and medical precision.

Section 2: Core Image Manipulation

2.1 Geometric Transformations

- **Resize:** Changing image dimensions. Necessary for standardizing inputs into Neural Network architectures.
- **Crop:** Extracting a Region of Interest (ROI). Essential for focusing on objects and removing background clutter.
- **Rotate:** Adjusting the orientation. Crucial for data augmentation to help models recognize tilted objects.

- Flip: Creating mirror images (horizontal or vertical) to increase dataset diversity without collecting new data.
- Affine Transformations: Maintaining parallel lines during stretching, shearing, or rotating.

2.2 Image Enhancement & Filtering

- Blur (Smoothing): Reducing noise and detail by averaging pixel neighborhoods. Types: Gaussian, Median, and Bilateral.
- Sharpen: Enhancing intensity differences to bring out edges and structural details.
- Thresholding: Converting a grayscale image to binary. Useful for segmenting objects from the background.
- Canny Edge Detection: A multi-stage pipeline for robust edge extraction. Includes blur, gradient calculation, and hysteresis.
- Morphological Operations: Shaping pixel structures via Dilation (adding pixels) or Erosion (removing pixels).

Section 3: Practical Programming (IPython Walkthrough)

Cell 1: Initialization and Image Loading

Production systems start with robust imports. We use OpenCV for logic and Matplotlib for notebook rendering.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load in BGR
img = cv2.imread('input.jpg')
# Standard Architect practice: Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
# Load grayscale for logic
img_gray = cv2.imread('input.jpg', 0)
```

Cell 2: Advanced Resizing Techniques

Resizing requires careful selection of interpolation methods to preserve image fidelity.

```
# Fixed-dimension resize for CNNs
resized = cv2.resize(img, (224, 224), interpolation=cv2.INTER_AREA)
# Scaling factor resize
scaled = cv2.resize(img, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_LINEAR)
```

Cell 3: Cropping (ROI) and Rotation

Cropping is achieved via NumPy slicing, while rotation utilizes an Affine transformation matrix.

```
# ROI Slicing [y1:y2, x1:x2]
h, w = img.shape[:2]
crop = img[h//4:3*h//4, w//4:3*w//4]

# Affine Rotation Matrix
center = (w // 2, h // 2)
M = cv2.getRotationMatrix2D(center, 45, 1.0)
rotated = cv2.warpAffine(img, M, (w, h))
```

Cell 4: Production Edge Detection Pipeline

Canny edge detection is sensitive to noise; always apply a Gaussian blur before the Canny function.

```
# 1. Noise Reduction
blur = cv2.GaussianBlur(img_gray, (5, 5), 0)
# 2. Edge Extraction
edges = cv2.Canny(blur, 50, 150)
# 3. Output
plt.imshow(edges, cmap='gray')
plt.show()
```

Cell 5: Automated Face Extraction Logic

Architecting a real-world tool that detects faces and standardizes them for a downstream GenAI model.

```
cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_frontalface_default.xml')
faces = cascade.detectMultiScale(img_gray, 1.1, 5)

for (x, y, w, h) in faces:
    face_roi = img[y:y+h, x:x+w]
    face_final = cv2.resize(face_roi, (128, 128))
    cv2.imwrite('output_face.jpg', face_final)
print(f'Processed {len(faces)} faces.')
```